

Fermion TeX Engine 実装総覧

常駐増分 TeX ランタイムと v3.2 完全一致システム

TeX DOM Runtime プロジェクト

2026 年 7 月 3 日

概要

本書は Fermion TeX Engine (リポジトリ `fermion-tex-engine`) の実装全体を一冊にまとめた技術報告である。本エンジンは TeX を「毎回ファイルから PDF を作るバッチ処理系」から「文書状態を常駐保持し、ソース差分から表示差分を生成する組版ランタイム」へ作り替えたものであり、1 打鍵あたり数ミリ秒の実組版で印刷同等のライブプレビューを実現する。v3.2 では「ライブプレビューが本物の `lualatex` 出力と原理的に一致する」ことを設計目標に据え、実測で全検証文書の全行が 0.02 bp 以内 (約 1/3600 mm) で一致することを自動検証基盤により証明した。本書では 3 世代のバックエンド構成、fork チェックポイント機構、実メイン垂直リスト収穫、TeX ページビルダーの忠実移植、検証基盤、および開発中に発見した罫を記す。

目次

1	目的と全体像	2
1.1	3 世代のバックエンド	2
1.2	変更していないもの	2
2	checkpoint バックエンドの基礎	3
2.1	fork は TeX 状態の完全なスナップショットである	3
2.2	ガレー抽出 — ノードリストから描画へ	3
2.3	収束の自己検証	3
3	v3.2 完全一致システム	4
3.1	診断 — 揺らぎはどこから来ていたか	4
3.2	実メイン垂直リスト収穫 (MVL harvest)	4
3.3	ページビルダー — <code>tex.web</code> と <code>ltoutput</code> の忠実移植	5
3.4	収束機構の拡張 — ブロック間レイアウト状態	6
3.5	精密レンダー層 (チャンク) の修正	6
4	検証基盤 — 「一致」を仕組みで証明する	6

4.1	基線比較器 verify-layout	6
4.2	増分経路の検証 verify-edits	7
4.3	結果	7
5	今回追加された罫アーカイブ	7
6	性能	8
7	既知の制限と今後	8

1 目的と全体像

出発点の要求は一言でいえば次のとおりである。

TeX を「毎回ファイルから PDF を作るバッチ処理系」ではなく、Web ブラウザのように文書状態を常駐保持し、ソース変更差分から表示差分を生成できる組版ランタイムへ作り替える。

成功条件は二段階で引き上げられてきた。第一段階は「この 1 文字変更で何が dirty になるか」にエンジンが正確に答え、変わったページだけをパッチすること。第二段階 (v3.2) は「プレビューに表示されるものが、同じソースを素の `lualatex` でコンパイルした PDF と原理的に一致する」ことである。行分割・改ページ・フロート配置・脚注・グルーの一本まで、場当たりの視覚調整ではなく、同じ計算が同じ入力に対して走る構造で一致させる。

1.1 3 世代のバックエンド

リポジトリには設計の進化がそのまま 3 つのバックエンドとして残っており、サーバー起動時に選択される (環境変数 `TDOM_BACKEND`)。

	v0: internal	v1: lualatex	v3: checkpoint (既定)
組版の実体	自前 JS (近似)	本物の <code>lualatex</code>	本物の <code>lualatex</code>
実行形態	常駐 JS オブジェクト	編集ごとに新プロセス	常駐プロセスツリー
1 編集の速さ	~1ms	~430ms	組版 4-14ms
品質	デモ水準	印刷同等	印刷同等 (一致を実証)
脚注/フロート/目次	なし	近似	ライブで実配置

v3 の核心は、`lualatex` そのものに常駐してもらい、POSIX の `fork(2)` (コピーオンライト) で「各ブロック境界時点の完全な TeX 状態」をプロセスとして凍結保存することである。編集されたら古い続きに基づくチェックポイントを殺し、直前のスナップショットから変わったブロックだけを組み直す。プロセス起動もフォーマットロードもフォントロードも発生しない。

1.2 変更していないもの

「TeX を改造した」のではない。`lualatex` (LuaHBTeX) のバイナリ、Knuth-Plass 行分割、数式組版、ハイフネーション、フォント処理 (`luaotfload/HarfBuzz`)、LaTeX カーネル、パッケージ群 (`amsmath`, `tikz`, `booktabs`, `luatexja`, ...) はすべて無改変である。外から注入するのは (1) `fork` を Lua に公開する 71 行の C シム、(2) ソケット通信とノードリスト走査を行う Lua デーモン、(3) `\label` 等を「元の動作を実行した上で記録する」約 40 行の TeX マクロシム、の 3 種だけである。「`lualatex` が計算する内容」は一切変えず、「いつ・どの単位で計算するか」と「結果をどこへ出すか」だけを差し替えた、というのが本プロジェクトの正確な要約である。

2 checkpoint バックエンドの基礎

2.1 fork は TeX 状態の完全なスナップショットである

TeX の内部状態（数万のマクロ、catcode 表、フォント、カウンタ、ボックスレジスタ）をアプリケーションレベルで保存・復元するのは絶望的だが、OS の視点ではただのプロセスメモリである。fork 自体は実測 0.21 ms、複製コストは書き換わったページ分だけ。文書を先頭からブロックごとに組版しながら各境界で fork して親を凍結すると、 ckpt_k = 「ブロック 1.. k を組版し終えた時点の TeX 完全状態」のプロセス鎖ができる。

デーモンの凍結は 1 つの末尾再帰マクロで実現される：

```
\def\TDOMloop{\directlua{tdom_wait()}\TDOMloop}
```

親は `tdom_wait()` 内のブロッキング read で永久停止し（CPU は食わない）、fork の子だけが Lua から return して TeX の入力へ戻る。その入力とは直前に `tex.print` で注入されたブロック組版コードであり、組版・報告を終えた子は再び `tdom_wait()` に入って次のチェックポイントとして凍結する。

2.2 ガレー抽出 — ノードリストから描画へ

組み上がったノードリストは Lua デーモンが歩き、「どのフォントのどの文字をどの座標に置くか」を JSON でオーケストレータ (Node.js) へ送る。最重要の設計判断は **run (グリフ連なり) を kern / glue の出現ごとに分割すること**である。TeX は単語内カーニングを kern ノードとして実体化するので、run 内部にはもう位置調整が存在せず、「開始 x だけ送れば、あとはフォントの字送り幅を足すだけで TeX と同じ座標になる」ことが保証される。ブラウザは TeX が使ったフォントファイルそのものを `@font-face` で受け取り、カーニング・リガチャ再適用を無効化して描く。これにより 1 グリフごとの座標を送らずに済む。

2.3 収束の自己検証

増分計算の正しさは通常「依存の完全列挙」に懸かるが、TeX では catcode もマクロも動的であり完全列挙は不可能である。本エンジンの停止判定はより強い性質を持つ：「**クリーンなはずのブロックを実際にもう 1 つ組版してみて、出力 (galley ハッシュ) と状態ベクトル (追跡カウンタ等の出口値) が前回と完全一致したら止まる**」。停止判定そのものが検算になっており、LaTeX の「もう一回コンパイル」をブロック単位に局所化した形である。

3 v3.2 完全一致システム

3.1 診断 — 揺らぎはどこから来ていたか

v3.1 まではブロックを `\vbox{...}` の中で組版していた。このグループ境界で TeX の状態が失われ、それを JS 側の近似式で「再構成」していたことが、実 PDF との「行数や位置の微妙な違い」の根本原因だった。具体的には：

1. **ブロック間グルーの推定**： $\max(\text{lineskip}, \text{baselineskip} - d_{\text{prev}} - h) + \text{parskip}$ という等価式で段落間の行送りを再構成していたが、実際の `\prevdepth` 連鎖・`\addvspace`・`\parskip` と微妙にずれ、ページ内で累積した。
2. **@afterheading 状態の喪失**：見出し直後の `\everypar`（インデント抑制）や `club penalty 10000` が `vbox` のグループ境界で消えた。目次（`\section*` が `@nobreak` を立てたまま終わる）の直後では、次の `\section` の `\if@nobreak` 分岐が実際と逆に転び、見出し前の `\addvspace` 約 15bp が出たり出なかったりした。
3. **ページビルダーが近似**：`\topskip` 未実装（毎ページ約 3pt 容量がずれ、境界ケースで行数が揺れる）、`widow/club penalty` の無視、`\footnoterule` を固定 6bp と近似（実際は正味 0pt）、`ins.h+2` のようなマジックナンバー等。

対策は「JS で垂直方向の寸法を発明しない。すべての値を TeX の実ノードまたは実測パラメータから取り、消費するアルゴリズムは TeX 自身のものを移植する」ことである。

3.2 実メイン垂直リスト収穫 (MVL harvest)

v3.2 ではブロックを **本物のメイン垂直リスト (MVL) 上で組版する**。TeX 内蔵のページビルダーは次の 3 点で「休眠」させる：

- `\vsize=\maxdimen` — ページは決して満ちず、出力ルーチンは自然には発火しない。
- `\holdinginserts=1` — 脚注の `ins` ノードが行位置のままストリームに残る。
- **ダミーボックス** — 起動時に `\hbox to 0pt{}` を一度だけ寄与させてページビルダー内部の `page_contents` フラグを `box_there` にし（このフラグは Lua から到達できない）、以後は Lua で `tex.lists.page_head` のリストだけをダミーに差し替える。これにより「空ページ先頭でのグルー破棄」と「`\topskip` グルーの挿入」が永久に抑止される。どちらも実ページ境界でオーケストレータ側が行う仕事である。

ブロックの組版後、`tdom_report()` がページリストを収穫し、新しいダミーを播種してからチェックポイントとして凍結する。この方式の意味は決定的である：`\prevdepth`・`\everypar`・`\spacefactor` などのブロック間状態が **fork のプロセス連鎖として自然に継続**するため、収穫されるノード列は「素の `lualatex` が文書全体を一気に組んだときの MVL」と **バイト単位で一致**する。段落間グルー・`\parskip`・`\addvspace`・`widow/club penalty`（見出し直後の $10000 + 150 = 10150$ まで）がすべて TeX の実物になる。この同一性は、素のコンパイルとの対照実験（`\directlua` 収穫呼び出しがノード

列に痕跡を残さないことの確認)で検証した。

強制排出(`\newpage·\clearpage·生`の`\penalty-10000`)だけは出力ルーチンを発火させる。安全網`tdom_absorb_output()`が`\box255`の中身を寄与リスト経由でページに戻し、`tdom:eject:penalty` マーカーを植えるので、オーケストレータのページビルダーがその位置で正確に改ページする(`\clearpage`の`-10001`なら`\@docclearpage`と同様に空ページを破棄しフロートを放流する)。

なお入力途中の未閉ブレースは`\long`マクロ引数のランナウェイとなり、注入した報告コードごと EOF まで食い尽くして fork 子を殺す(旧 vbox 方式では閉じ括弧が構造的に止めていた)。オーケストレータが不均衡分の`}`を自動補完してから注入し、均衡が戻れば厳密経路に自動復帰する。

3.3 ページビルダー — tex.web と ltoutput の忠実移植

`pagebuilder.js` は v3.2 で全面書き換えされ、TeX のページ分割アルゴリズム(`tex.web` §1005–1008)と LaTeX の出力ルーチン(`ltoutput`)の transcription となった。このファイルの中に発明された寸法は存在しない。すべての値はストリーム(実ノード)か、ライブ TeX から伸縮成分込み(`\gluestretch`等)で実測したパラメータである。

■改ページ判定 TeX と同じ合法ブレイクポイント(非破棄物直後の`glue`、`glue`が続く`kern`、 $p < 10000$ の`penalty`)、同じコスト関数 $c = b + p$ (`badness` b は`tex.web` §108の整数アルゴリズムをそのまま実装)、同じ「最良ブレイクを記憶し、`overfull`で発火、コストのタイはあとが勝つ」規則。`widow/club/\@secpenalty`はストリーム内の実`penalty`として自然に効く。`\topskip`(ページ先頭ボックス上の実グルー)、`\maxdepth`の深さ繰入れ、`footins`のページゴール減算(`\skip\footins`の自然長と各`ins`の高さ $\times \text{\count\footins}/1000$)も`tex.web`のとおりである。

■フロート `\@addtocurcol` $\rightarrow$ `\@addtotoporbot` $\rightarrow$ `\@addtobot`、ページ境界での`\@startcolumn`(`\@tryfcolumn`のフロートページ生成、`\@addtonextcol`)、`\@makefcolumn`放流、型順序保存の衝突規則まで、`latex.ltx`の条件式を逐語的に移植した。カウンタ(`topnumber`系)と分数(`\topfraction`系)、分離グルー(`floatsep`系)、フロートページの`\@fptop/\@fpsep/\@fpbot`はすべて実行時に実測送信される。`h`配置は本文ストリームへの`[pen][intextsep][box][pen][intextsep]`注入で、垂直モード宣言のフロートには`\vskip-\parskip`も再現する。

■ページ組立(`\@makecol`) 本文 + `\vskip\skip\footins` + クラスの`\footnoterule`を実測したカーン／罫アイテム列として再生(articleなら`kern -3pt·罫 0.4pt·kern 2.6pt = 正味 0pt`) + `ins`内容の連結、`\@combinefloats`の上下フロート、末尾の`\vskip-\dp`(最終深さ打消し)と`\@textbottom`(`raggedbottom`なら`0pt plus .0001fil`)。最後に`vbox-to-\@colht`相当のグルー分配(次数別`stretch/shrink`、`fil`優先)で絶対座標を確定するため、`raggedbottom` / `flushbottom` / フロートページの`fil`配分がTeXと同じ算術で出る。文書末尾は`\enddocument`の`\clearpage`(`\vfil` + 排出 + フロート放流)を再現する。これにより最終ページの下部フロートがページ底に沈む挙動まで一致する。

3.4 収束機構の拡張 — ブロック間レイアウト状態

MVL 方式では「次のブロックの先頭グルー」が前のブロックの出口状態 (`\prevdepth` と `\if@nobreak`) に依存する。そこでこの 2 つを状態ベクトルに追加した (`tdom@pd`, `tdom@nobreak`)。既存の「もう 1 ブロック組んで一致するまで」の自己検証がそのまま拡張され、目次の固定点反復や後方参照パスがブロック連鎖を短絡してもレイアウト状態の食い違いが自動的に検出・再組版される。すべての再組版経路は共通の `#retypesetChain` (収束するまで延長) を通る。

3.5 精密レンダー層 (チャンク) の修正

グリフで描けないもの (TikZ 等の `pdf_literal`、レガシー Computer Modern 数式) は「即時はグリフ近似 → 本物の PDF 由来 SVG に自動置換」の 2 層戦略をとる。v3.2 ではこの層の不具合を 4 件修正した：

1. **フロートチャンク URL の #**：チャンク鍵 `b14#1` がそのまま URL に入り、フラグメント扱いで 404 になっていた (プレビューで図が壊れアイコンになる主因)。 `encodeURIComponent` で解決。
2. **PDF フラッシュ前の変換競合**：`finish_pdffile` コールバック (DONE 通知) はディスクフラッシュ前に発火するため、`pdftocairo` が不完全な PDF を読むことがあった (チャンクがランダムに欠落)。 `%%EOF` 確認まで待機して解決。
3. **hyperref 系文書の分離レンダ**：常駐ツリーが PDF を出荷できないプリアンブルでは単発 `lualatex` でチャンクを作るが、これが旧 `vbox` 方式のままでガレーとずれていた。休眠ページ方式を単発実行でも再現し (`tdom:isostart` マーカー、前ブロックの実 `\prevdepth`・`\if@nobreak` を注入)、ガレーと同じ先頭グルーを持つ画素整合チャンクを得る。
4. **luatexja の shipout フック**：`ltjs` 系クラスではページ寸法指定が上書きされ、チャンクが紙サイズで出ることがある。箱は `\hoffset=-1in` により常に原点にあるため、SVG の `viewBox` を既知の箱寸法へ正規化する `cropSvg` で常に正確に切り出す。

4 検証基盤 — 「一致」を仕組みで証明する

4.1 基線比較器 `verify-layout`

`tools/verify-layout.mjs` は同じソースに対して (a) エンジンヘッドレスで走らせた表示リストと、(b) 素の `lualatex` 2 パスコンパイルの PDF を突き合わせる。真値の抽出は **PDF コンテントストリームの直接パース** (`qpdf` で展開し、`1 0 0 1 x y Tm` 行列と TJ 配列を読む。 `tools/pdf-lines.mjs`) による。当初 `pdftocairo -svg` の座標を使っていたが、**pdftocairo は座標に約 0.1% の系統歪みを入れる**ことを手製 PDF で実証したため排除した (この歪みは許容誤差 0.1bp の 3 倍以上あり、偽の不一致を量産していた)。

比較はページ数・行数 (ベースラインのクラスタ)・各行の y (0.02bp 量子化)・行頭 x を対象とし、

精密チャンクに覆われる領域は「チャンクの画素 = PDF の画素」なので被覆判定で処理する。

4.2 増分経路の検証 verify-edits

tools/verify-edits.mjs は open 直後ではなく、**編集を重ねた後の状態**が正確であることを検証する：語の書換えと取消し、数式環境の挿入（式番号の連鎖再採番）、ラベル改名と参照更新、段落置換の 6 編集を増分機構で適用し、最終ソースを新規に 2 パスコンパイルした真値と比較する。

4.3 結果

検証文書	一致行	備考
samples/demo-lua.tex	85/85	脚注・[t] フロート・TikZ・目次・文献
templates/minimal.tex	3/3	
templates/article-en.tex	62/62	[t] 図・[b] 表・脚注・文献
templates/article-ja.tex	43/43	luatexja 日本語
templates/math-notes.tex	50/50	定理環境・整列数式
ユーザー実文書 (ltjsarticle+hyperref)	36/36	maketitle・目次・\clearpage
編集 6 連続後（増分経路）	86/86	verify-edits

すべての行で $\Delta y \leq 0.02$ bp（検証器の量子化粒度）。既存の統合テスト 30 本もすべて通過している。ブラウザ実機でも実 PDF のラスタと並べて目視比較し、TikZ 図・表・脚注罫・数式・ノンプルの位置と見た目の一致を確認した。

5 今回追加された罣アーカイブ

開発中に発見し、将来の変更者が再び踏まないよう記録しておくべき事実：

1. **pdftocairo は座標を歪める**：-svg 出力はコンテンツ座標に約 0.1% のスケール誤差を含む（手製の最小 PDF で実証）。検証には PDF コンテンツストリームを直接読むこと。
2. **page_contents フラグは Lua から見えない**：tex.lists.page_head をいくら差し替えても、ページビルダーが「ページは空」と思っていればグルー破棄と topskip 挿入が起こる。実ボックスを一度 TeX 経由で寄与させてフラグを立ててからリスト手術を行うこと。
3. **未閉ブレースはランナウェイで子を殺す**：\emph{ のような入力途中のブロックは \long 引数走査が注入コードごと EOF まで消費する。注入前に括弧収支を数えて自動閉鎖する。
4. **finish_pdffile はディスクフラッシュ前に発火する**：fork 子の stdio バッファは _exit 時にしか書き切られない。ファイル末尾の %%EOF を確認してから変換にかけること。
5. **luatexja は shipout をフックする**：ページ寸法の直前指定が無効化されることがある。出荷箱を原点に固定 (\hoffset=-1in) し、viewBox 切り出しで吸収する。
6. **チャンク鍵の # は URL で死ぬ**：フロートチャンク鍵 (blockId#n) を URL に埋めるときは必ずエンコードする。

6 性能

v3.2 の完全一致化は増分性能を損なっていない。編集 1 回のフォアグラウンド経路は従来どおり「fork (〜0.2ms) + 変更段落の Knuth–Plass (数 ms) + ノード走査と JSON」であり、実測で組版 4–14ms、打鍵から画面反映まで約 30ms。ページビルダーの移植は純 JS 演算で 1ms 未満、チェックポイントのスパース化 (最大 64 常駐) やページの参照同一性による表示リスト再利用もそのまま機能している。チャンク (TikZ 等) は従来どおり非同期で 1–2 秒後に精密画像へ置換される。

7 既知の制限と今後

- 脚注のページまたぎ分割 (`\vsplit` 相当) は未実装。該当ケースは検出して診断に出す。
- 二段組 (twocolumn)・`\marginpar`・`\enlargethispage` は未対応。
- `figure/table` 以外のフロート環境 (パッケージ独自の `\newfloat` 系) はシム対象外で、出力ルーチン発火時は安全網が吸収するのみ。
- Windows ネイティブは `fork(2)` 依存のため対象外 (WSL は可)。

いずれも「未対応であることが分かる」形で残してあり、場当たりの近似でごまかす箇所は残していない。今後対応する場合も本書の方針——実ノードと実測パラメータのみを入力とし、TeX 自身のアルゴリズムを移植し、`verify-layout` で全行一致を確認する——に従うこと。

本書自体も検証対象と同じ `lualatex` (LuaHBTeX, TeX Live 2024) で組版されている。